

Рефакторинг коду та проведення Code Review за індустріальними стандартами

Мета роботи

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

Результати Code Review одnogрупника

№	Рядок коду	Проблема	Категорія	Рекомендація
1	78	Число 3000 є магічним числом	Магічні числа	Створити MAX_FREE_BITRATE_KBPS = 3000
2	122	Логіка створення об'єкта HistoryRecord, додавання його до списку self.history та виклик методу .save_locally() повторюється двічі з єдиною відмінністю у значенні operation_type ("PROCESS_SUCCESS" / "PROCESS_FAILED")	Дублювання коду	застосувати операцію Consolidate Duplicate або Extract Method: винести цю повторювану логіку в окремий приватний метод класу User, наприклад: def _add_history_record(self, file: MediaFile, op_type: str) -> None
3	20	Назва аргументу format збігається з назвою вбудованої функції Python format(), через що оригінальна функція перекривається всередині цього методу	Погане іменування	перейменувати параметр та властивість класу на file_format
4	15	Число 30 означає тривалість підписки у днях, але записане як жорсткий літерал	Магічні числа	Створити SUBSCRIPTION_DURATION_DAYS = 30

5	30	Розмір буфера завантаження (5 МБ) зашитий прямо всередині методу. Якщо знадобиться змінити розмір шматка для оптимізації мережі або написання тестів, зробити це буде важко.	Магічні числа	Створити DEFAULT_CHUNK_SIZE
---	----	--	---------------	-----------------------------

Посилання на PR з коментарями Code Review (URL).

<https://github.com/OlehSheremet/bse-lr1-Sheremet/pull/3#issue-4596854857>

Результати рефакторингу власного коду

Рефакторинг 1: Усунення "Поганого іменування"

БУЛО:

```
class MediaFile:

    def init(self, file_name: str, file_size: int, format: str):

        self.file_name = file_name

        self.file_size = file_size

        self.format = format
```

СТАЛО:

```
class MediaFile:

    def init(self, file_name: str, file_size: int, file_format: str):

        self.file_name = file_name

        self.file_size = file_size

        self.format = file_format # Властивість залишаємо, міняємо лише параметр
```

ЧОМУ:

Використання зарезервованих слів або вбудованих функцій як назв змінних є антипатерном (shadowing). Це може призвести до непередбачуваної поведінки, якщо

всередині методу знадобиться використати стандартну функцію `format()`. Зміна на `file_format` робить код безпечнішим.

Рефакторинг 2: Усунення "Магічних чисел"

БУЛО:

```
class Subscription:
    def activate(self) -> None:
        # ...
        self.expiry_date = datetime.date.today() + datetime.timedelta(days=30)
```

СТАЛО:

```
class Subscription:

    DURATION_DAYS = 30 # Додали константу рівня класу

    # ...

    def activate(self) -> None:

        # ...

        self.expiry_date = datetime.date.today() +
            datetime.timedelta(days=self.DURATION_DAYS)
```

ЧОМУ:

Число 30 було магічним, його призначення не було очевидним без вчитування в контекст `timedelta(days=...)`. Винесення його в іменовану константу `DURATION_DAYS` робить конфігурацію тривалості підписки централізованою. Якщо бізнес-логіка зміниться (наприклад, підписка стане 31 день), достатньо буде змінити значення лише в одному місці.

Рефакторинг 3: Усунення "Дублювання коду"

БУЛО:

у методі `batch_process_files` класу `User` код `record = HistoryRecord(...), self.history.append(...), record.save_locally()` повторювався у блоці `try` та `except`

СТАЛО:

```
def _add_history_record(self, file: MediaFile, op_type: str) -> None:
```

```
record = HistoryRecord(
    record_id=len(self.history) + 1,
    timestamp=datetime.datetime.now(),
    operation_type=op_type,
    media_file=file
)

self.history.append(record)

record.save_locally()
```

#Всередині try (замість 7 рядків):

```
self._add_history_record(file, "PROCESS_SUCCESS")
print("-----")
```

#Всередині except (замість 7 рядків):

```
self._add_history_record(file, "PROCESS_FAILED")
print("-----")
```

ЧОМУ:

Застосовано операцію Extract Method. Це усунуло порушення принципу DRY (Don't Repeat Yourself). Тепер логіка створення запису історії зосереджена в одному місці, що значно спрощує метод `batch_process_files` і полегшує подальшу підтримку.

The screenshot shows a VS Code editor with a Python script named `main.py` and its test results. The script defines a `Subscription` class with methods for initialization, activation, and expiration. The test results show that all tests passed.

```
class Subscription:
    DURATION_DAYS = 30

    def __init__(self, sub_id: int, status: str, expiry_date: datetime.date):
        self.sub_id = sub_id
        self.status = status
        self.expiry_date = expiry_date

    def activate(self) -> None:
        if self.status == "ACTIVE":
            print(f"Підписка {self.sub_id} вже активна.")
            return
        self.status = "ACTIVE"
        self.expiry_date = datetime.date.today() + datetime.timedelta(days=self.DURATION_DAYS)
        print(f"Підписка {self.sub_id} успішно активована до {self.expiry_date}.")

# --- Кінець MainApp ---
if __name__ == "__main__":
    # Test session starts
    platform = sys.platform
    rootdir = C:\Users\corycap\vscode-l1-sheremet
    plugins = cov-7.1.0
    collected 16 items

    src/test/tests.py ..... [100%]

    ===== 16 passed in 0.05s =====

> |
```

Порівняння метрик

Цикломатична складність:

До:

The screenshot shows a VS Code editor with a Python script named `main.py` and its test results. The script defines a `Subscription` class with methods for initialization, activation, and expiration. The test results show that all tests passed.

```
class Subscription:
    DURATION_DAYS = 30

    def __init__(self, sub_id: int, status: str, expiry_date: datetime.date):
        self.sub_id = sub_id
        self.status = status
        self.expiry_date = expiry_date

    def activate(self) -> None:
        if self.status == "ACTIVE":
            print(f"Підписка {self.sub_id} вже активна.")
            return
        self.status = "ACTIVE"
        self.expiry_date = datetime.date.today() + datetime.timedelta(days=self.DURATION_DAYS)
        print(f"Підписка {self.sub_id} успішно активована до {self.expiry_date}.")

# --- Кінець MainApp ---
if __name__ == "__main__":
    # Test session starts
    platform = sys.platform
    rootdir = C:\Users\corycap\vscode-l1-sheremet
    plugins = cov-7.1.0
    collected 16 items

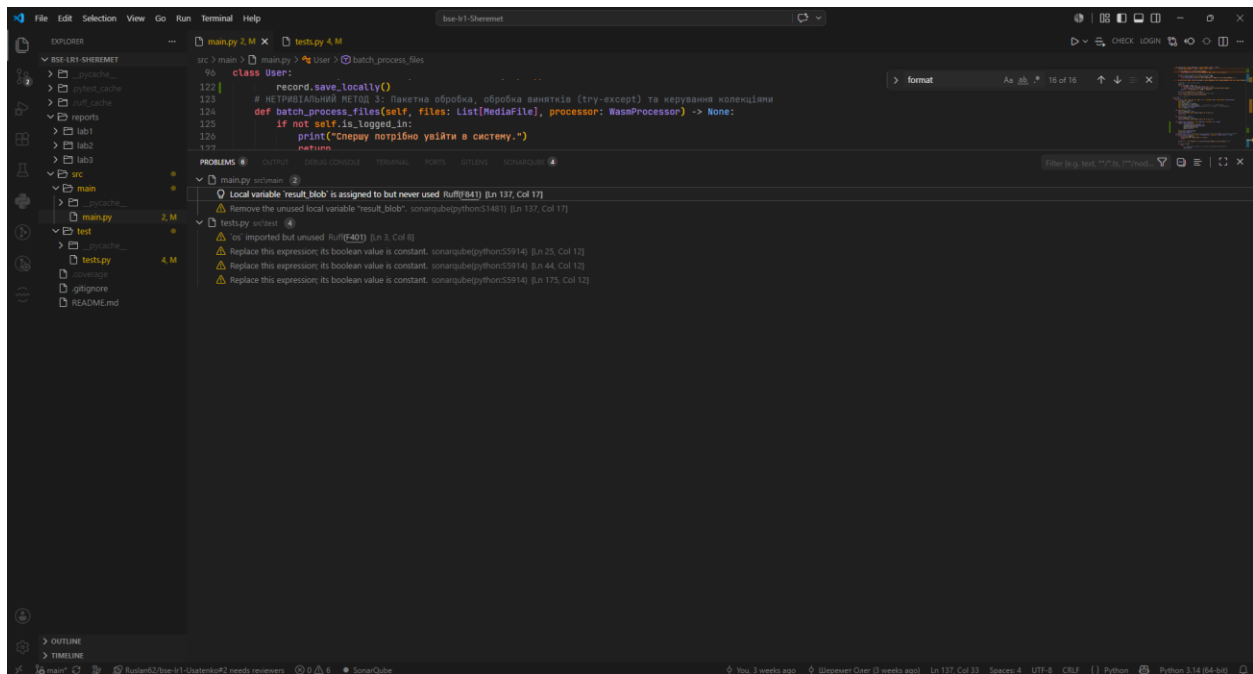
    src/test/tests.py ..... [100%]

    ===== 16 passed in 0.05s =====

> |
```

Після:

Після:



Підсумкова рефлексія

Найціннішим досвідом під час виконання роботи стало практичне проведення Code Review з використанням GitHub Pull Requests. Я навчився ефективно ідентифікувати типові проблеми якості коду (code smells), такі як магічні числа, дублювання коду та інші. Особливо корисним було самостійно провести рефакторинг: застосування операції Extract Method дозволило зосередити логіку в одному місці, що значно полегшує подальшу підтримку коду та робить його безпечнішим. Крім того, наочне порівняння аналітичних метрик підтвердило успішність виконаної роботи завдяки зменшенню кількості зауважень від аналізаторів SonarLint та Ruff.